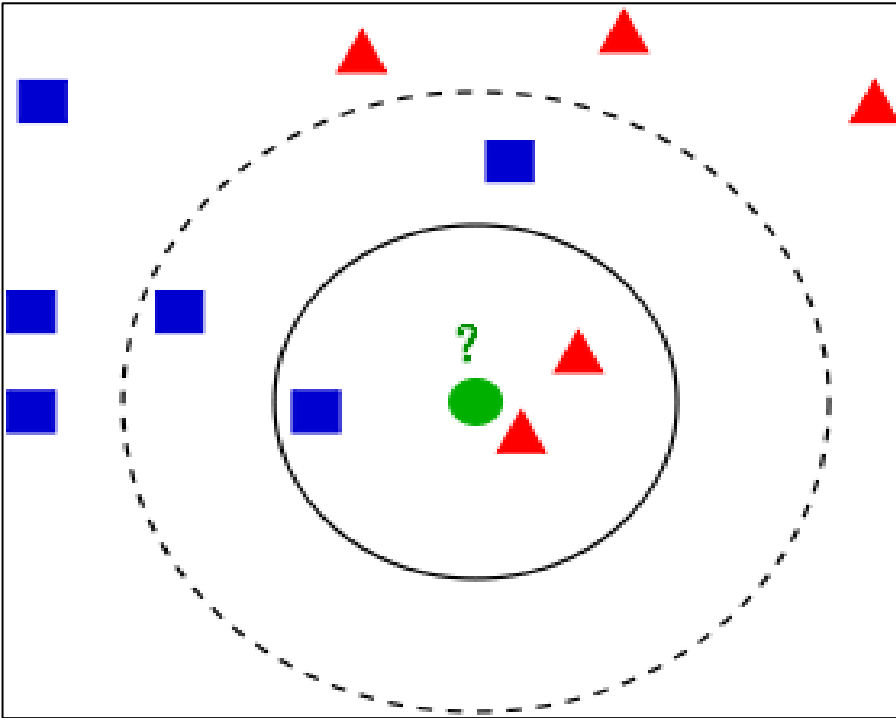


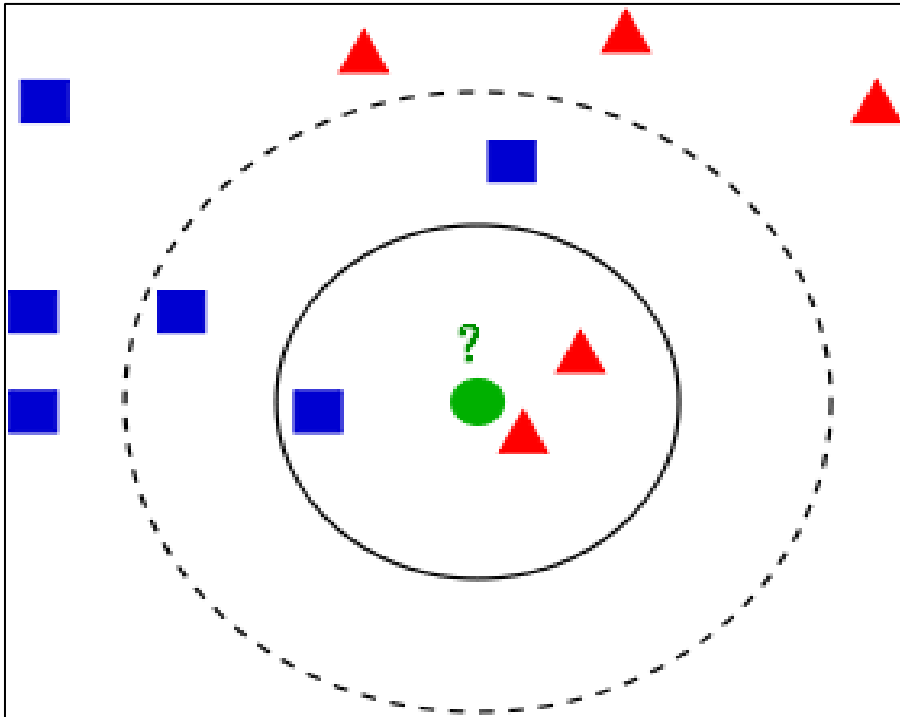
K vecinos mas cercanos
(KNN)

Concepto del KNN



1. Etiquetar cada patrón de entrenamiento al número de clase --- datos etiquetados
2. El patrón de entrada se clasifica acuerdo a clase de **K** vecinos más cercanos

Ejemplo



Resultado de clasificación cambia dependiendo del valor de K

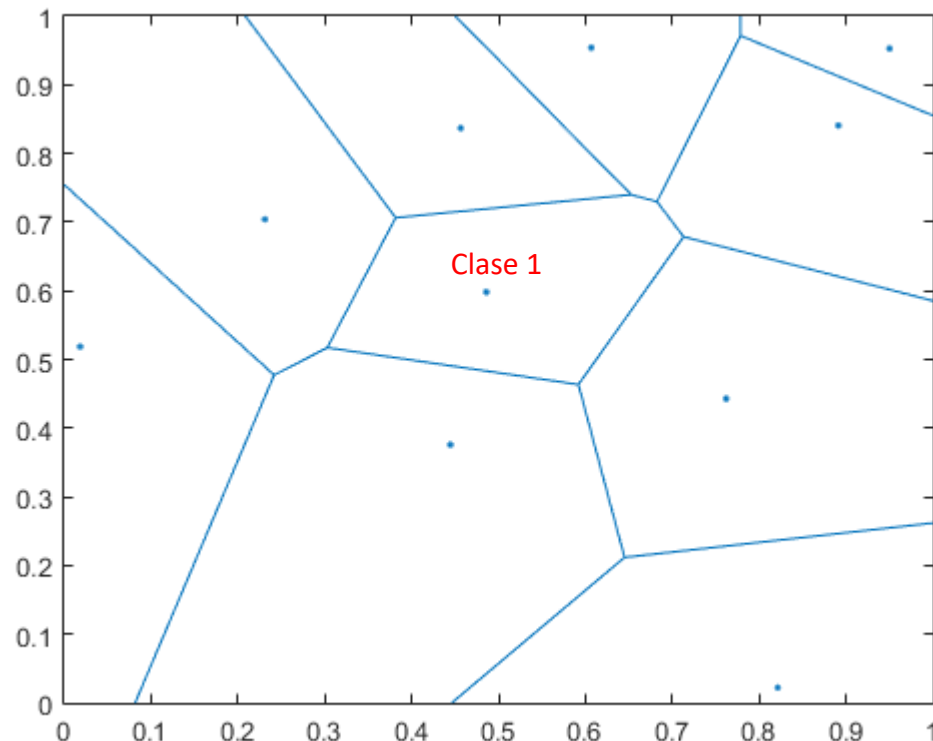
Cuando $K > 1$, aplica voting (decisión por mayoría)

K=1:  $\rightarrow$ 

K=3:   vs   $\rightarrow$ 

K=5:   vs     $\rightarrow$ 

Partición de Voronoi (Voronoi Partición)



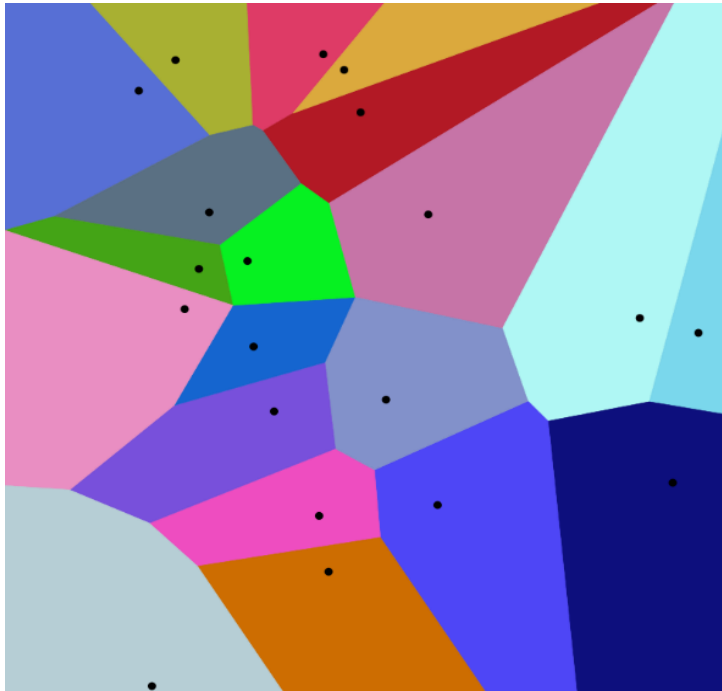
Distancia Euclidiana

Cuando $K=1$, el espacio de características se puede dividir usando Partición de Voronoi.

- Cada punto azul indica un patrón con una clase predeterminada.
- El patrón de entrada que cae dentro del polígono, se asigna la misma clase del punto azul.
- La construcción de partición de Voronoi depende de métrica de distancia.

En K-NN, la métrica de distancia es muy importante

Partición de Voronoi con diferentes métricas de distancia



Distancia Euclidiana



Distancia Manhattan (o City block)

Métricas de distancia

1. Distancia Euclidiana
2. Distancia Euclidiana Estandarizada
3. Distancia de Manhattan (City Block)
4. Distancia de Chebychev
5. Distancia Minkowski
6. Distancia Coseno
7. Distancia de Mahalanobis
8. Distancia de Hamming
9. Distancia de Jaccard

Distancia Euclidiana

En espacio de características

$$\mathbf{X} = [x_1, x_2, x_3, \dots, x_{n-1}, x_n], \quad \mathbf{Y} = [y_1, y_2, y_3, \dots, y_{n-1}, y_n]$$

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + \dots + (x_{n-1} - y_{n-1})^2 + (x_n - y_n)^2}$$

$$d^2 = (\mathbf{X} - \mathbf{Y})(\mathbf{X} - \mathbf{Y})^T$$

elementos	a	b	c
X_1	4.8	72	3.5
X_2	8.1	73	2.1
X_3	4.8	54	3.6
X_4	4.7	65	5.5
X_5	5.4	59	2.7
X_6	8.2	64	2.9
X_7	3.9	61	3.1
X_8	2.6	94	3.5
X_9	4.6	53	2.9
X_{10}	3.9	68	3.4
Promedio	5.1	66.3	3.32
Desviación Estándar	1.777	11.85	0.893

Preguntas

¿Característica X_1 es mas cerca a X_2 o X_3 ?
(intuitivamente)

$$\begin{aligned} &\text{Distancia entre } X_1 \text{ y } X_2 \\ d &= \sqrt{(4.8 - 8.1)^2 + (72 - 73)^2 + (3.5 - 2.1)^2} \\ &d=3.7216 \end{aligned}$$

$$\begin{aligned} &\text{Distancia entre } X_1 \text{ y } X_3 \\ d &= \sqrt{(4.8 - 4.8)^2 + (72 - 54)^2 + (3.5 - 3.6)^2} \\ &d=18.0 \end{aligned}$$

¿Distancia euclidiana está representando correctamente la distancia de dos característica?

elementos	a	b	c
X_1	4.8	72	3.5
X_2	8.1	73	2.1
X_3	4.8	54	3.6
X_4	4.7	65	5.5
X_5	5.4	59	2.7
X_6	8.2	64	2.9
X_7	3.9	61	3.1
X_8	2.6	94	3.5
X_9	4.6	53	2.9
X_{10}	3.9	68	3.4
Promedio	5.1	66.3	3.32
Desviación Estándar s	1.777	11.85	0.893

Distancia Euclidiana estandarizada

$$d = \sqrt{\frac{1}{s_1^2}(x_1 - y_1)^2 + \frac{1}{s_2^2}(x_2 - y_2)^2 + \dots + \frac{1}{s_n^2}(x_n - y_n)^2}$$

$$s_i^2 = \frac{1}{n - 1} \sum_{j=1}^M (x_{ij} - u_i)^2$$

Formula con vector

$$d^2 = (X - Y)V^{-1}(X - Y)^T$$

$$V = \begin{bmatrix} s_1^2 & 0 & 0 & 0 & 0 \\ 0 & s_2^2 & 0 & 0 & 0 \\ 0 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & s_{n-1}^2 & 0 \\ 0 & 0 & 0 & 0 & s_n^2 \end{bmatrix}$$

elementos	a	b	c
X_1	4.8	72	3.5
X_2	8.1	73	2.1
X_3	4.8	54	3.6
X_4	4.7	65	5.5
X_5	5.4	59	2.7
X_6	8.2	64	2.9
X_7	3.9	61	3.1
X_8	2.6	94	3.5
X_9	4.6	53	2.9
X_{10}	3.9	68	3.4
Promedio	5.1	66.3	3.32
Desviación Estándar s	1.777	11.85	0.893

Distancia Euclidiana estandarizada

$$d_{std} = \sqrt{\frac{1}{s_1^2}(x_1 - y_1)^2 + \frac{1}{s_2^2}(x_2 - y_2)^2 + \dots + \frac{1}{s_n^2}(x_n - y_n)^2}$$

$$s_i^2 = \frac{1}{n-1} \sum_{j=1}^M (x_{ij} - u_i)^2$$

$$V = \begin{bmatrix} 3.16 & 0 & 0 \\ 0 & 140.42 & 0 \\ 0 & 0 & 0.80 \end{bmatrix}$$

Ejercicio: Calcular distancia Euclidiana estandarizada

$$d_{std}(X_1, X_2) , d_{std}(X_1, X_3)$$

$$d_{std}(X_1, X_2) = 2.4297$$

$$d_{std}(X_1, X_3) = 1.5231$$

Estandarización

$$X_{stand} = \frac{X - \text{promedio}(X)}{s(X)}$$

elemento	a	b	c
X_1	4.8	72	3.5
X_2	8.1	73	2.1
X_3	4.8	54	3.6
X_4	4.7	65	5.5
X_5	5.4	59	2.7
X_6	8.2	64	2.9
X_7	3.9	61	3.1
X_8	2.6	94	3.5
X_9	4.6	53	2.9
X_{10}	3.9	68	3.4
Promedio	5.1	66.3	3.32
s	1.777	11.85	0.893

elementos	a	b	c
X_1	-0.17	0.48	0.20
X_2	1.69	0.57	-1.37
X_3	-0.17	-1.04	0.31
X_4	-0.23	-0.11	2.44
X_5	0.17	-0.62	-0.69
X_6	1.74	-0.19	-0.47
X_7	-0.68	-0.45	-0.25
X_8	-1.41	2.34	0.20
X_9	-0.23	-1.12	-0.47
X_{10}	-0.68	0.14	0.09
Promedio	0	0	0
s	1	1	1

Concepto básico de métrica de City block (distancia Manhattan)



— Distancia Euclidiana
(distancia más corta)

— Distancia City block
(Distancia de Manhattan)

Distancia City block

elementos	a	b	c
X_1	-0.17	0.48	0.20
X_2	1.69	0.57	-1.37
X_3	-0.17	-1.04	0.31
X_4	-0.23	-0.11	2.44
X_5	0.17	-0.62	-0.69
X_6	1.74	-0.19	-0.47
X_7	-0.68	-0.45	-0.25
X_8	-1.41	2.34	0.20
X_9	-0.23	-1.12	-0.47
X_{10}	-0.68	0.14	0.09
Promedio	0	0	0
s	1	1	1

$$X = [x_1, x_2, x_3, \dots, x_{n-1}, x_n],$$

$$Y = [y_1, y_2, y_3, \dots, y_{n-1}, y_n]$$

$$d_{cb}(X, Y) = (|x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|)$$

$$= \sum_{i=1}^n |x_i - y_i|$$

Ejercicio: Calcular distancia City Block
 $d_{cb}(X_1, X_2)$, $d_{cb}(X_1, X_3)$

$$d_{cb}(X_1, X_2) = |-0.17 - 1.69| + |0.48 - 0.57| + |0.20 - (-1.37)|$$

$$= 3.52$$

$$d_{cb}(X_1, X_3) = |-0.17 - (-0.17)| + |0.48 - (-1.04)| + |0.20 - 0.31|$$

$$= 1.63$$

Distancia de Chebyshev

elementos	a	b	c
X_1	-0.17	0.48	0.20
X_2	1.69	0.57	-1.37
X_3	-0.17	-1.04	0.31
X_4	-0.23	-0.11	2.44
X_5	0.17	-0.62	-0.69
X_6	1.74	-0.19	-0.47
X_7	-0.68	-0.45	-0.25
X_8	-1.41	2.34	0.20
X_9	-0.23	-1.12	-0.47
X_{10}	-0.68	0.14	0.09
Promedio	0	0	0
s	1	1	1

$$\mathbf{X} = [x_1, x_2, x_3, \dots, x_{n-1}, x_n],$$

$$\mathbf{Y} = [y_1, y_2, y_3, \dots, y_{n-1}, y_n]$$

$$d_{ch}(\mathbf{X}, \mathbf{Y}) = \max(|x_1 - y_1|, |x_2 - y_2|, \dots, |x_n - y_n|)$$

Ejercicio

¿Cuál vector de la tabla tiene la distancia mas corta de X_1 usando la distancia de Chebychev?

Respuesta es X_{10}

Distancia de Chebyshev

elementos	a	b	c	Distancia Chebyshev
X_1	-0.17	0.48	0.20	0
X_2	1.69	0.57	-1.37	1.86
X_3	-0.17	-1.04	0.31	1.52
X_4	-0.23	-0.11	2.44	2.24
X_5	0.17	-0.62	-0.69	1.1
X_6	1.74	-0.19	-0.47	1.91
X_7	-0.68	-0.45	-0.25	0.93
X_8	-1.41	2.34	0.20	1.86
X_9	-0.23	-1.12	-0.47	1.6
X_{10}	-0.68	0.14	0.09	0.51
Promedio	0	0	0	
s	1	1	1	

Distancia de Minkowski

La distancia de Minkowski es una expresión general de varias métricas de distancia.

$$d = \sqrt[p]{\sum_{i=1}^n |x_i - y_i|^p}$$

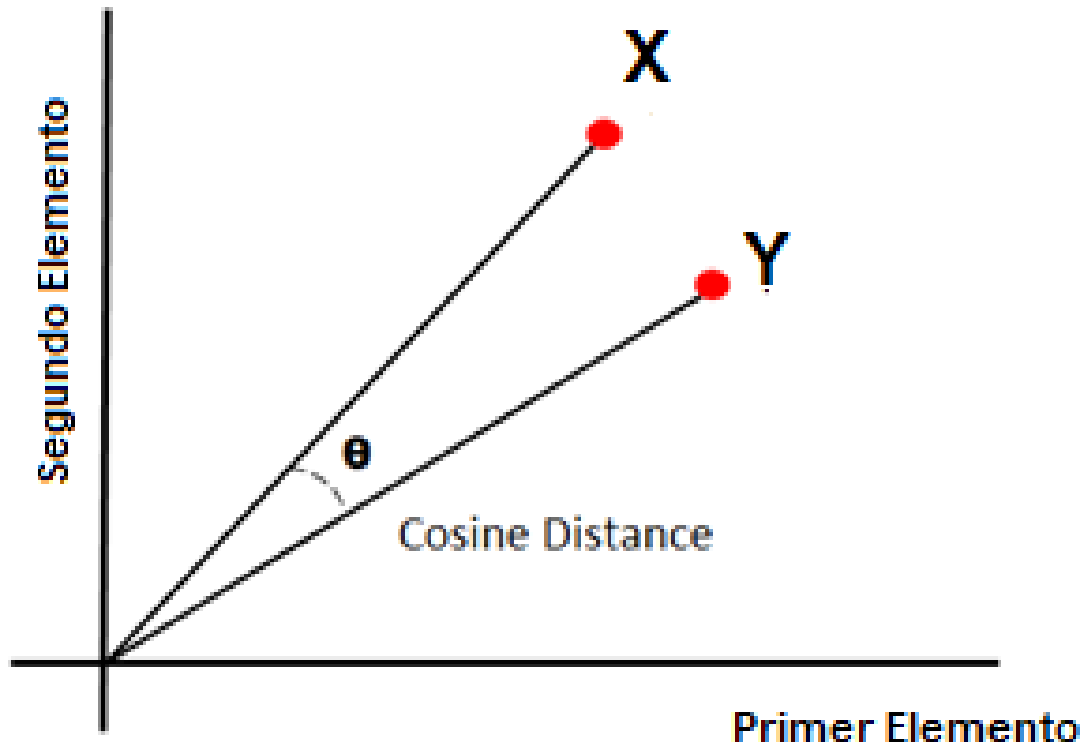
Dependiendo del valor p , se puede representar diferentes métricas de distancia

Pregunta: Cuando p tiene los siguientes valores. ¿La métrica de Minkowski en que métrica convierte?

$p = 1, p=2, p=\infty$

p	Métrica	Norma
1	City Block (Manhattan)	L_1
2	Euclidiana	L_2
∞	Chebychev	L_∞

Distancia coseno



$$X = [x_1, x_2, x_3, \dots, x_{n-1}, x_n],$$

$$Y = [y_1, y_2, y_3, \dots, y_{n-1}, y_n]$$

$$\text{Similitud de Coseo} = \cos(\theta) = \frac{\vec{X} \cdot \vec{Y}}{\|\vec{X}\| \|\vec{Y}\|}$$

$$= \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$

$$d_c = 1 - \text{similitud de cosenno}$$

Distancia Mahalanobis

La distancia Mahalanobis es algo similar con la distancia Euclidiana estandarizada, pero en lugar de dividir por un matriz diagonal que consiste en varianza de cada elemento, se divide por la matriz de covarianza.

Donde

$$MC = \begin{bmatrix} \sigma_1^2 & \sigma_{1,2} & \sigma_{1,3} & \cdots & \sigma_{1,n} \\ \sigma_{1,2} & \sigma_2^2 & \sigma_{2,3} & \cdots & \sigma_{2,n} \\ \sigma_{1,3} & \sigma_{2,3} & \sigma_3^2 & \cdots & \sigma_{3,n} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \sigma_{1,n} & \sigma_{2,n} & \sigma_{3,n} & \cdots & \sigma_n^2 \end{bmatrix}$$

$$\begin{aligned} \sigma_{i,j} &= \sigma_{j,i} \\ \sigma_i^2 &= \frac{1}{N-1} \sum_{k=1}^N (x_{i,k} - \mu_i)^2 \\ \sigma_{i,j} &= \frac{1}{N-1} \sum_{k=1}^N (x_{i,k} - \mu_i)(x_{j,k} - \mu_j) \end{aligned}$$

$$d_{mh} = \sqrt{(X - Y)MC^{-1}(X - Y)^T}$$

Distancia de Hamming

- La distancia de Hamming se usa generalmente cuando ambos vectores son binarios (0 o 1).
- La distancia es básicamente número de elementos desiguales dividido entre el número total de elementos.

$$d_{ham} = \frac{1}{n} |x_i \neq y_i|, i = 1..n$$

donde *|condición|* es el número de elementos que cumple la condición.

Ejercicio: Calcula la distancia de Hamming de los siguientes dos vectores A y B

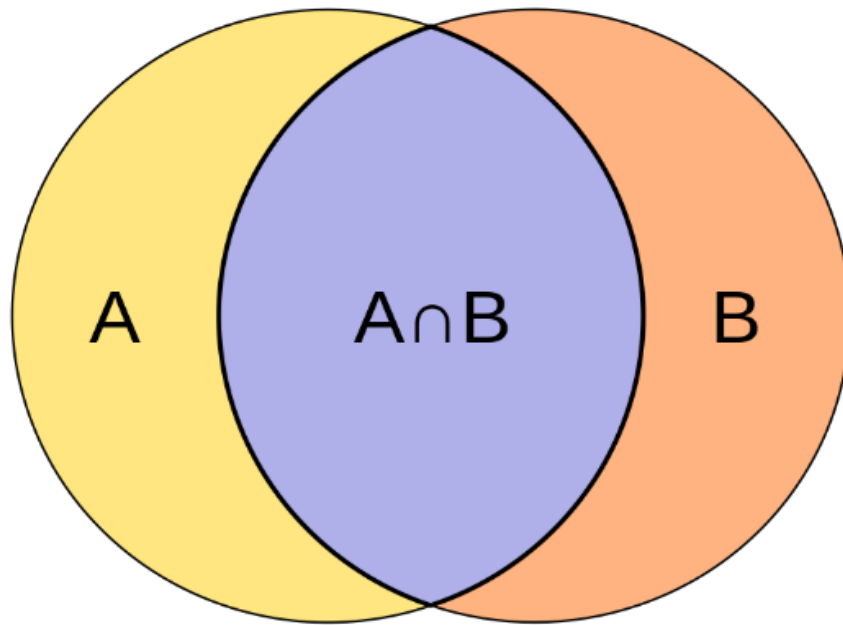
A=[1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0]

B=[1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1]

Respuesta: $d_{ham} = \frac{6}{11} = 0.545$

Distancia Jaccard

La distancia Jaccard está basada en el concepto de conjunto que involucra operación unión y intersección



Índice de Jaccard

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

distancia de Jaccard

$$d_J = 1 - J(A,B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

Se usa para segmentación de regiones para comparar con Ground Truth

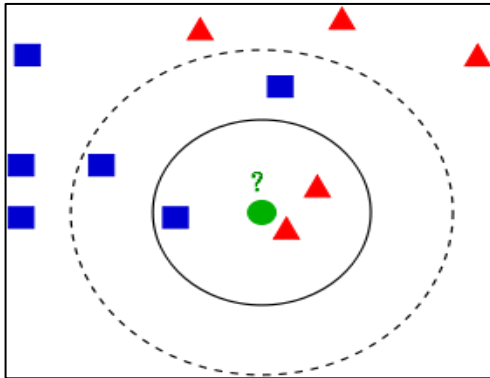
Implementación usando Sklearn

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, weights='uniform',  
algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None,  
n_jobs=None, **kwargs)
```

```
from sklearn.neighbors import KNeighborsClassifier
```

n_neighbors: Número de vecinos (5 es defaulto)

weights: Definir los pesos de vecinos. “uniform” o “distance” (uniform es defaulto)



Cuando `weights="uniform"` y `n_neighbors=5`, todos los 5 vecinos tienen mismo peso



La clase de dato de entrada es clase azul cuadro

Cuando `weights="distance"` y `n_neighbors=5`, todos los 2 vecinos triángulo rojo tienen mayor peso comparando con los tres cuadrados azul.

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, weights='uniform',  
algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None,  
n_jobs=None, **kwargs)
```

algorithm: Definir el algoritmo de búsqueda de K vecinos más cercanos

'auto', 'ball_tree', 'kd_tree', 'brute'

auto: *Depende del problema, selecciona algoritmo óptimo*

ball_tree , kd_tree: *algoritmos basado en búsqueda de árbol. ball_tree es eficiente para dimensión alta*

'brute': *búsqueda exhaustiva*

leaf_size: El número mínimo de elementos donde termina la búsqueda

p: Valor de p para métrica de minkowski (p=2)

metric: *Métrica de distancia (Default es 'minkowski')*



Default: Distancia Euclidiana

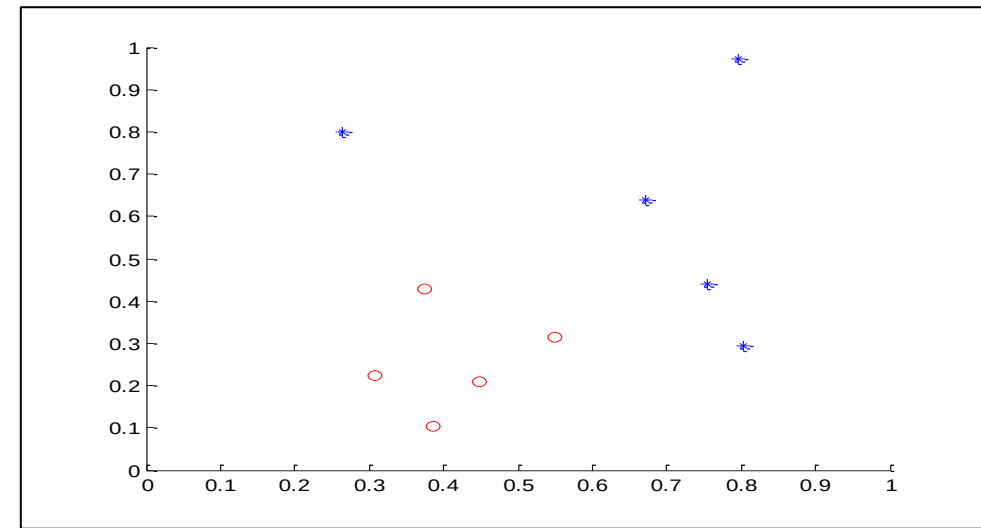
Árbol de K-dimensión (Kd Tree)

1. Seleccionar un vector de primera dimensión con el valor medio.
2. Particionar elementos en dos grupos
3. En cada grupo, seleccionar un vector de segunda dimensión con el valor medio.
4. Particionar elementos de cada grupo en dos grupos

Los procesos 1-4 iteran hasta que quede N elementos en cada nodo

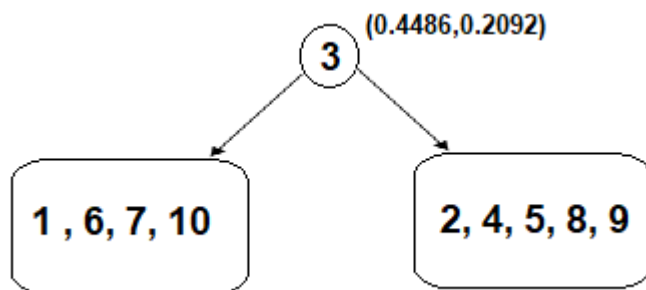
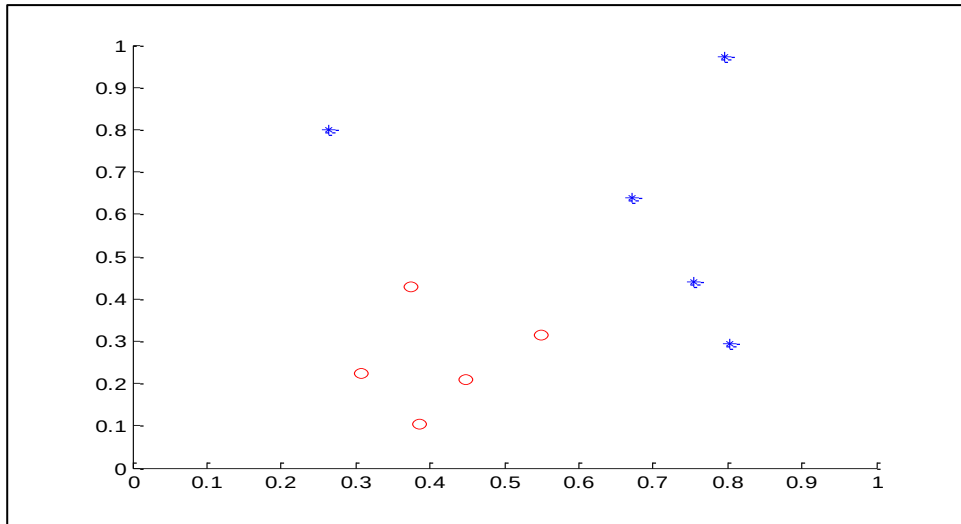
1. Seleccionar un vector medio en la 1ra dimensión

	x	y	etiqueta
1	0.3076	0.2238	0
2	0.8034	0.2950	1
3	0.4486	0.2092	0
4	0.6719	0.6378	1
5	0.7972	0.9736	1
6	0.2631	0.8002	1
7	0.3860	0.1024	0
8	0.7547	0.4397	1
9	0.5494	0.3140	0
10	0.3737	0.4279	0



[0.2631, 0.3076, 0.3737, 0.3860, 0.4486, 0.5494, 0.6719, 0.7547, 0.7972, 0.8034]

2. Particionar elementos en dos grupos



	x	y	etiqueta
1	0.3076	0.2238	0
2	0.8034	0.2950	1
3	0.4486	0.2092	0
4	0.6719	0.6378	1
5	0.7972	0.9736	1
6	0.2631	0.8002	1
7	0.3860	0.1024	0
8	0.7547	0.4397	1
9	0.5494	0.3140	0
10	0.3737	0.4279	0

3. En cada grupo, seleccionar un elemento con valor medio

	x	y	etiqueta
1	0.3076	 0.2238	0
2	0.8034	0.2950	1
3	0.4486	0.2092	0
4	0.6719	0.6378	1
5	0.7972	0.9736	1
6	0.2631	0.8002	1
7	0.3860	0.1024	0
8	0.7547	 0.4397	1
9	0.5494	0.3140	0
10	0.3737	0.4279	0

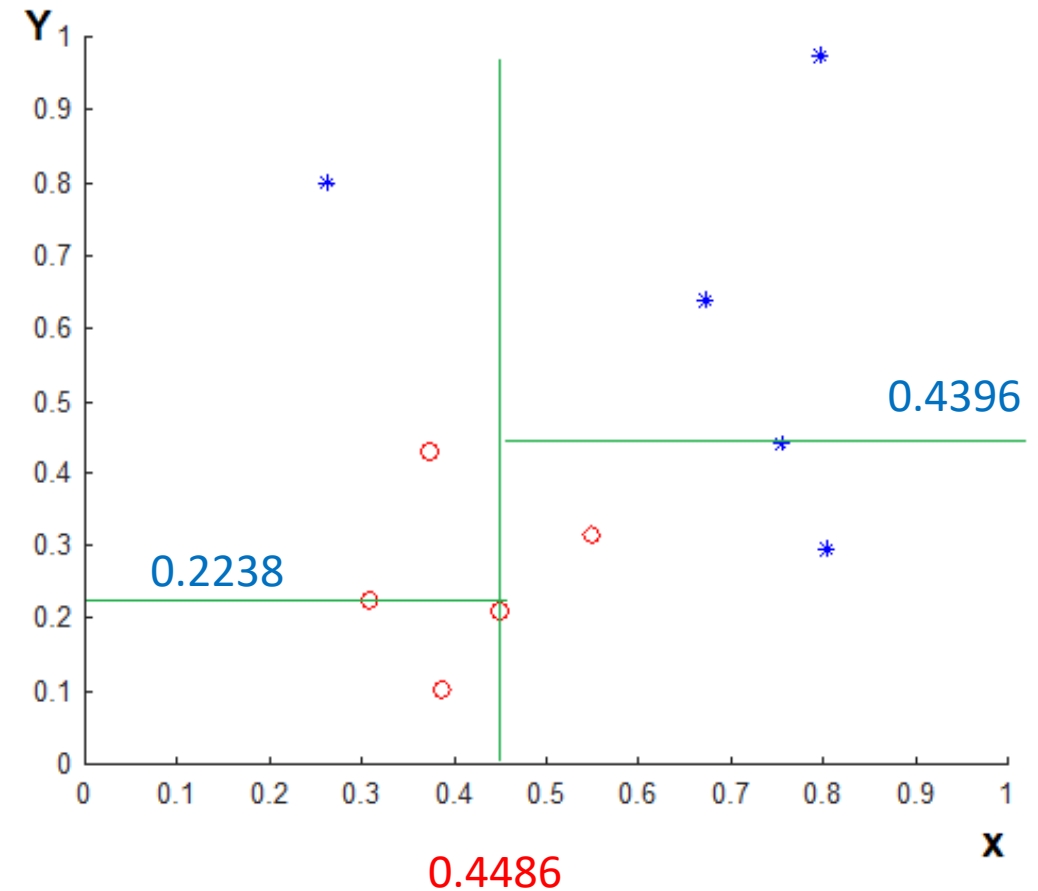
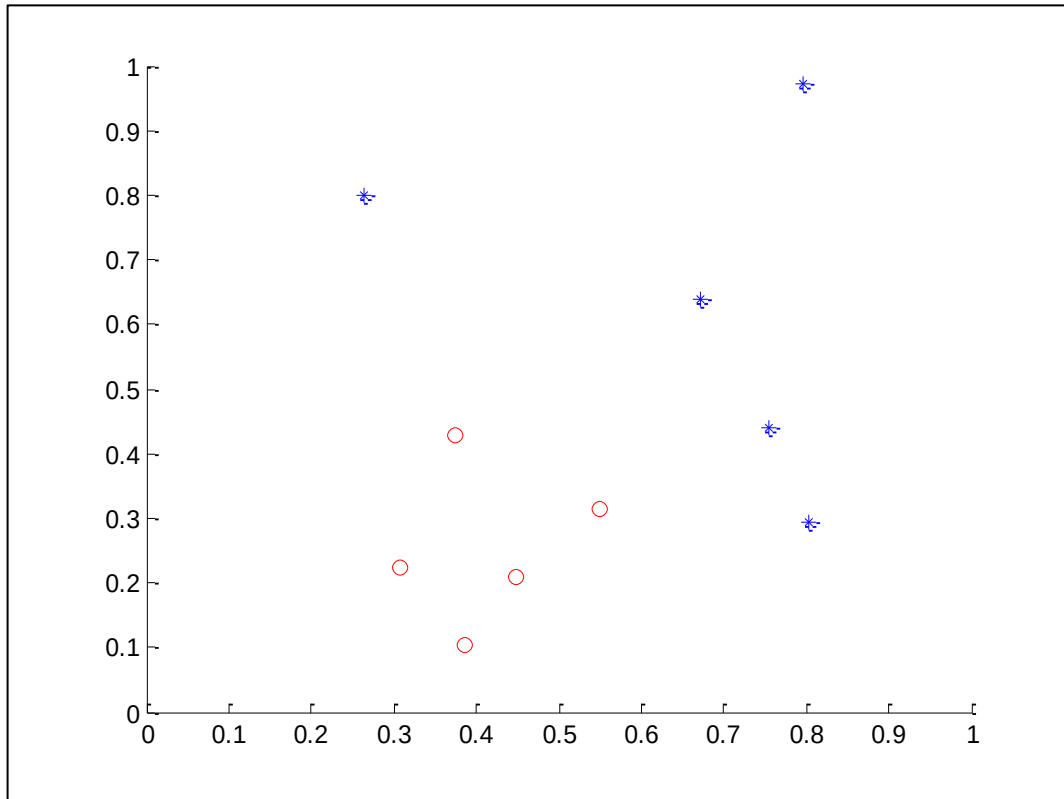
Grupo 1

[0.1024, 0.2238, 0.4279, 0.8002]

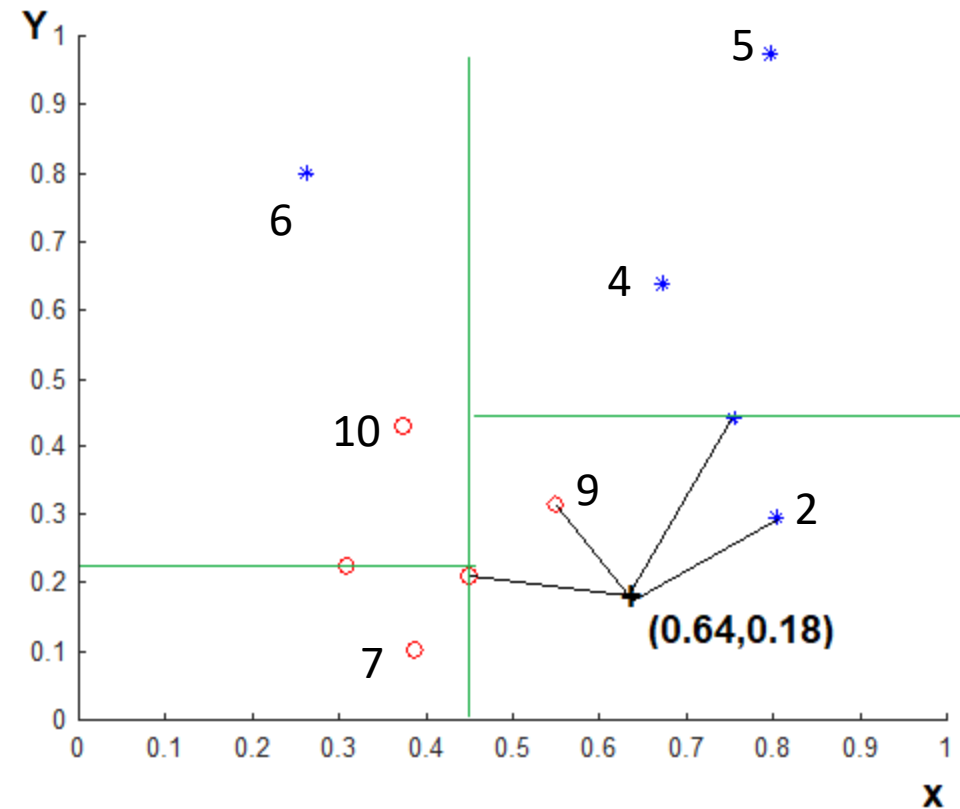
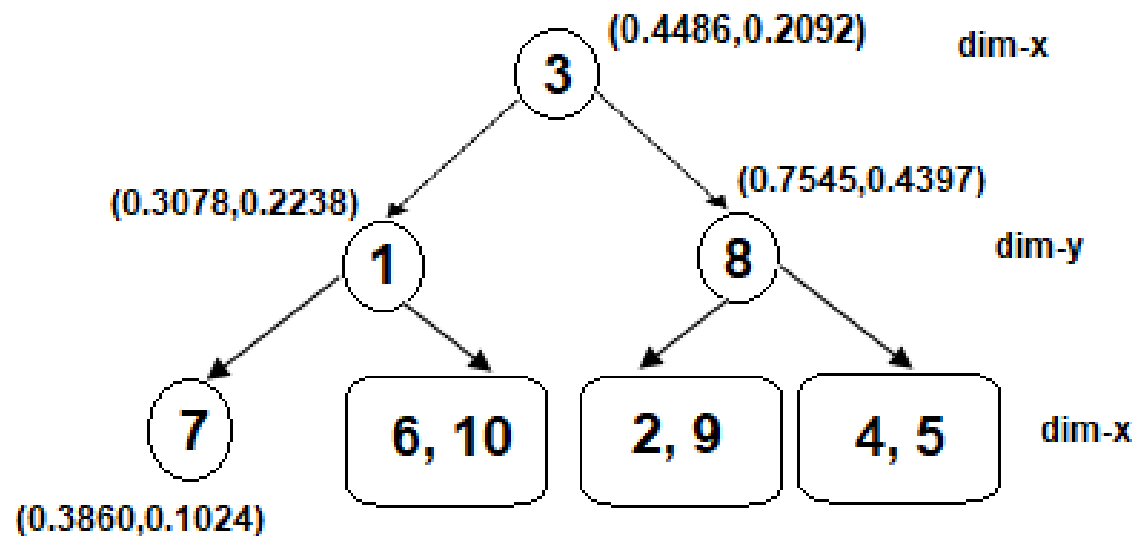
Grupo 2

[0.2950, 0.3140, 0.4397, 0.6378, 0.9736]

4. Particionar elementos en dos grupos (segundo dimensión)

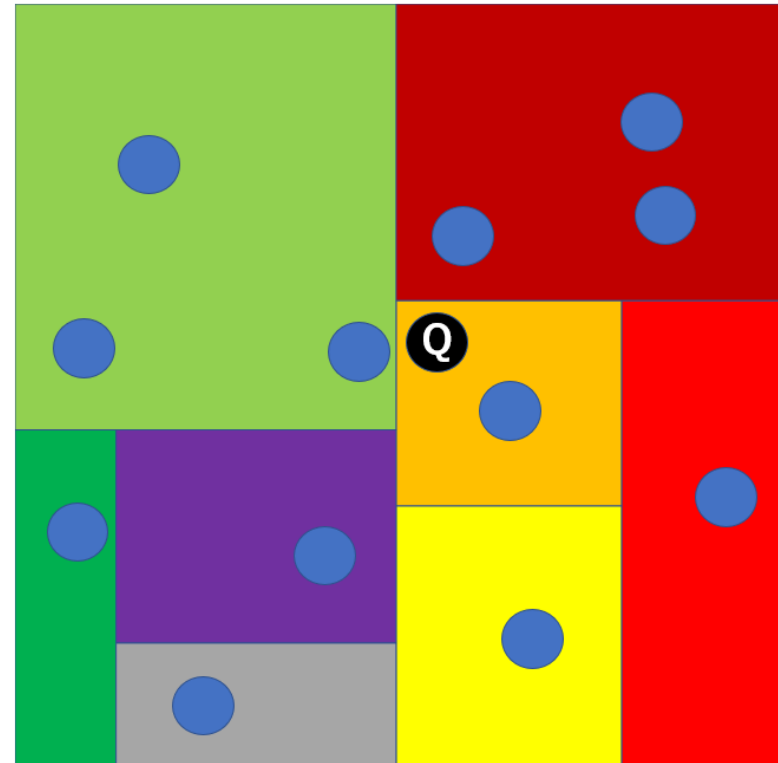
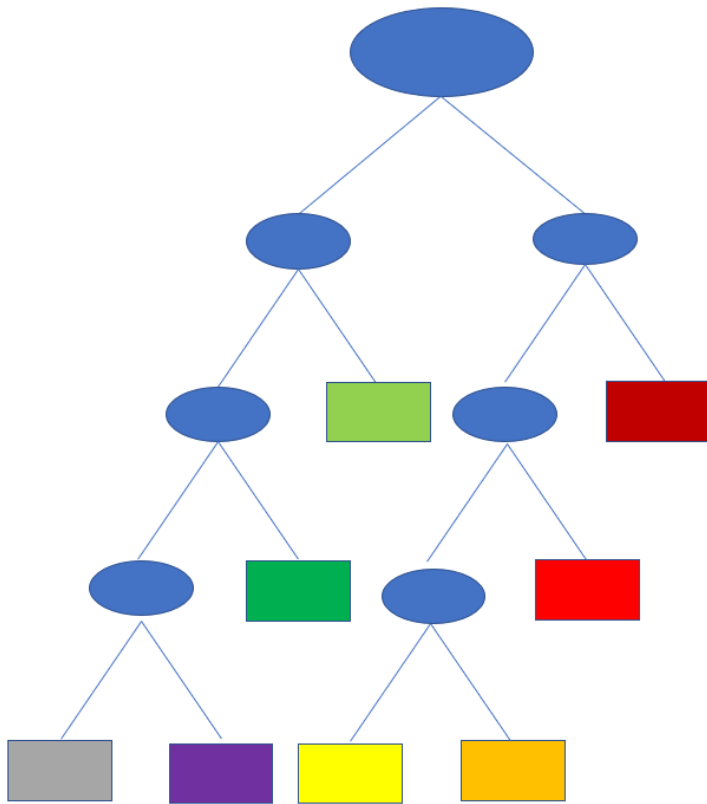


Búsqueda usando kd tree

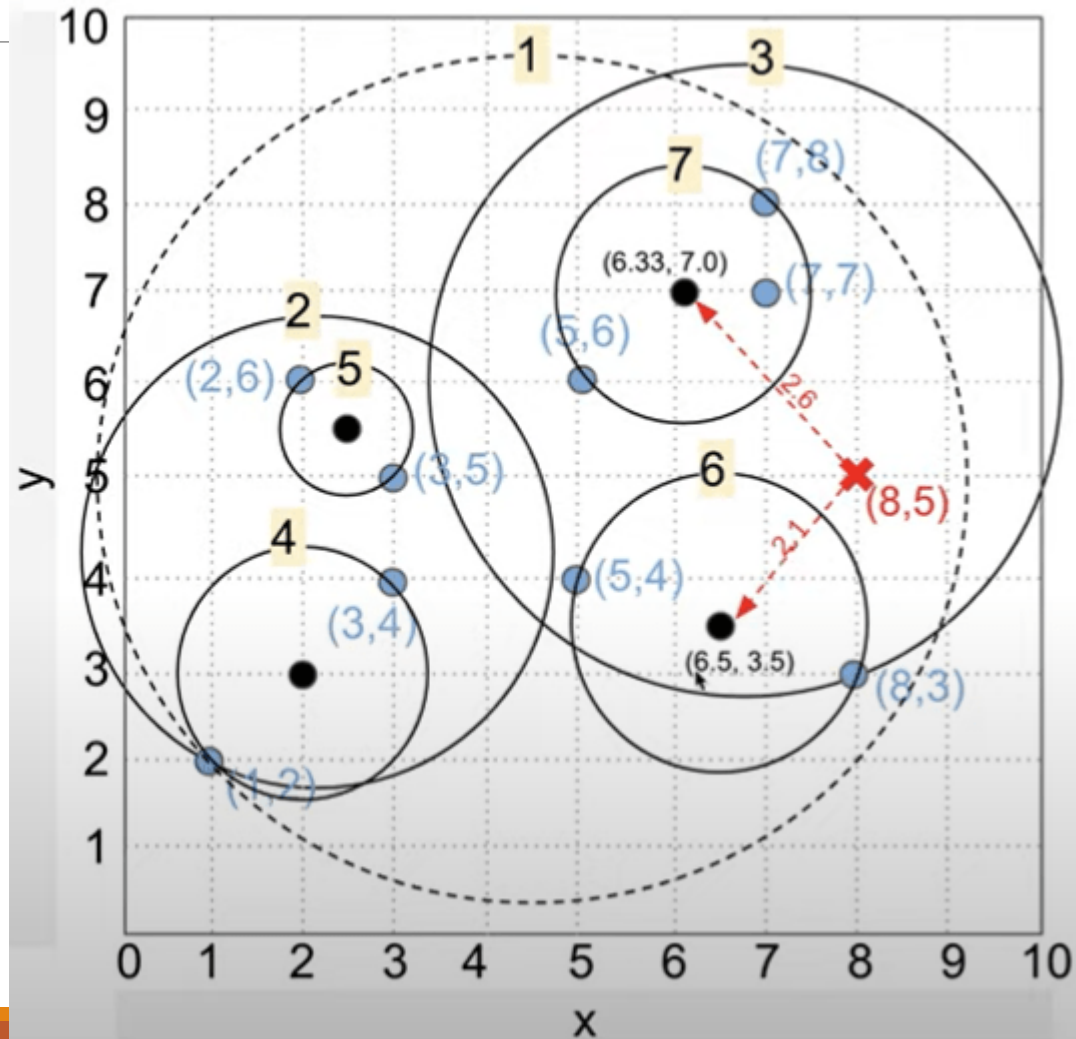


- Solamente calcular cuatro distancias con el elemento 2 y el elemento 9

Búsqueda usando kd Tree



Ball-Tree



Algoritmos de búsqueda basado en árbol

Algoritmo exhaustivo ↔ Algoritmo basado en árbol

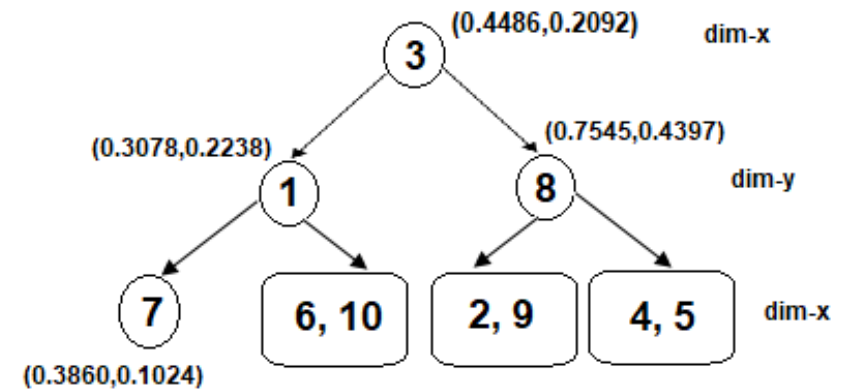
Comparación de número de operaciones (cálculo de distancia)

Datos total: 10 elementos

Número máximo de hojas en cada rama: 2

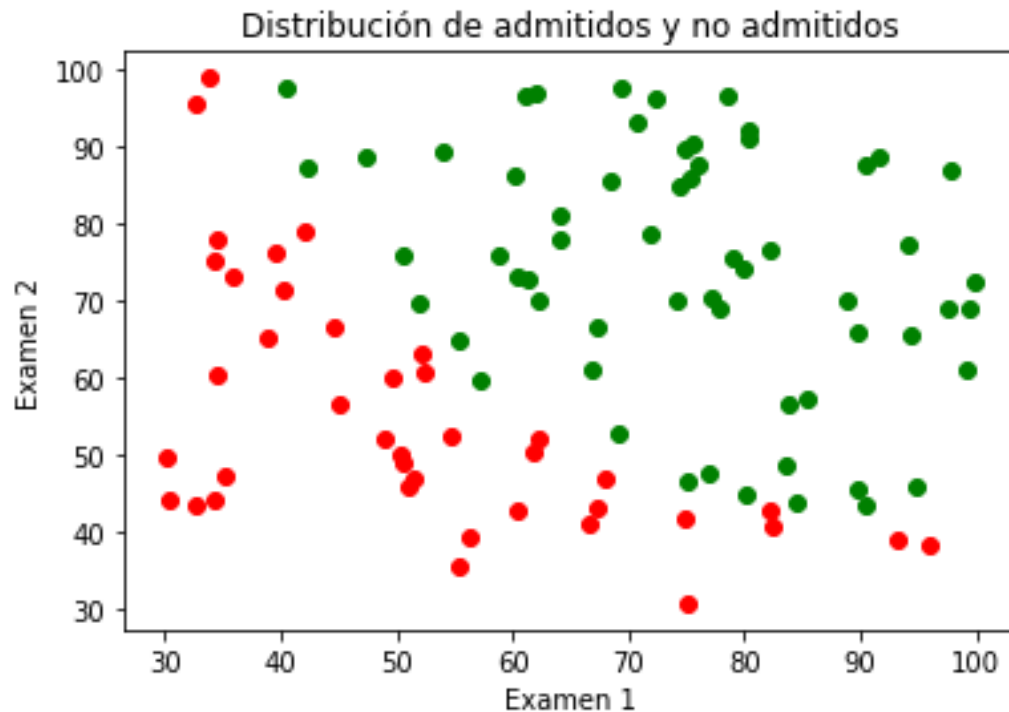
Algoritmos	Número de operaciones
Exhaustivo	?
Basado en árbol	?

Sin incluir construcción de árbol



Implementación 1

Datos de entrada: 100 datos con calificaciones de dos exámenes y resultado de admitido o no



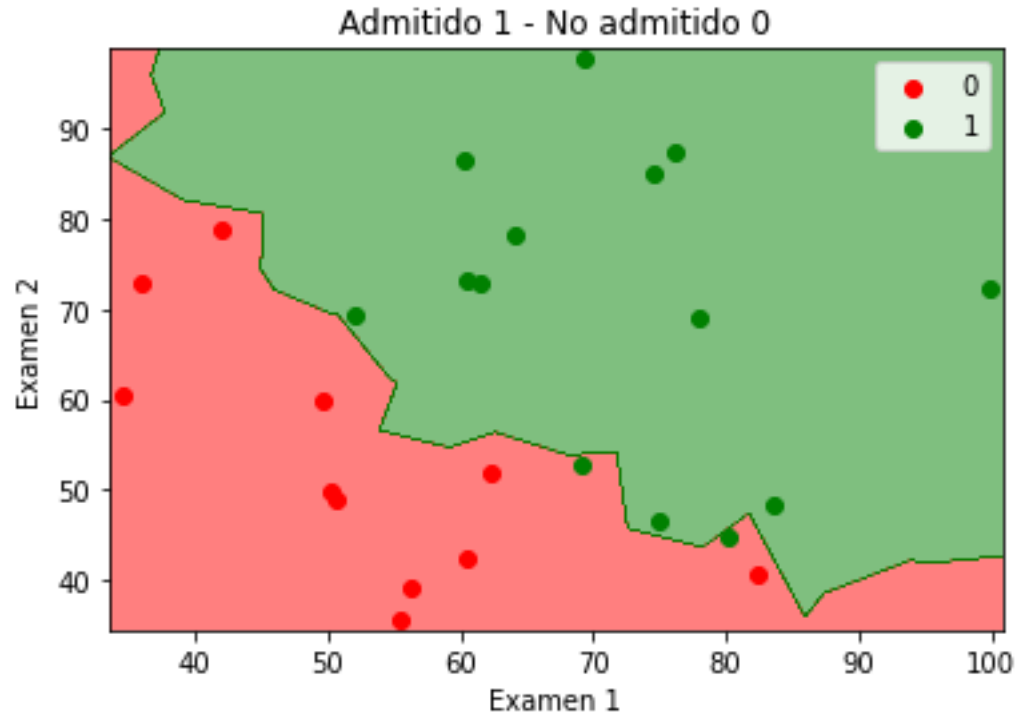
Cambiando el número de vecinos, analiza funcionamiento

Métrica de distancia: Euclidiano

Número de vecinos: 1, 3, 5 y 7

Resultados

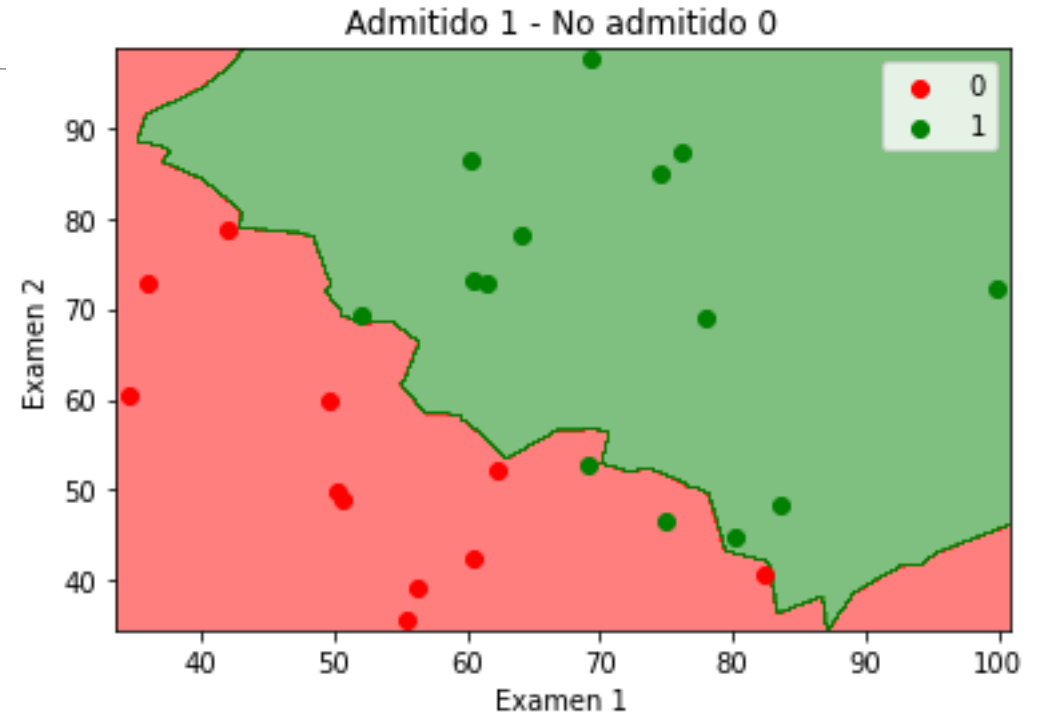
K=1 Accuracy=0.92



Predicho

GT	11	0
	2	12

K=3 Accuracy=0.92

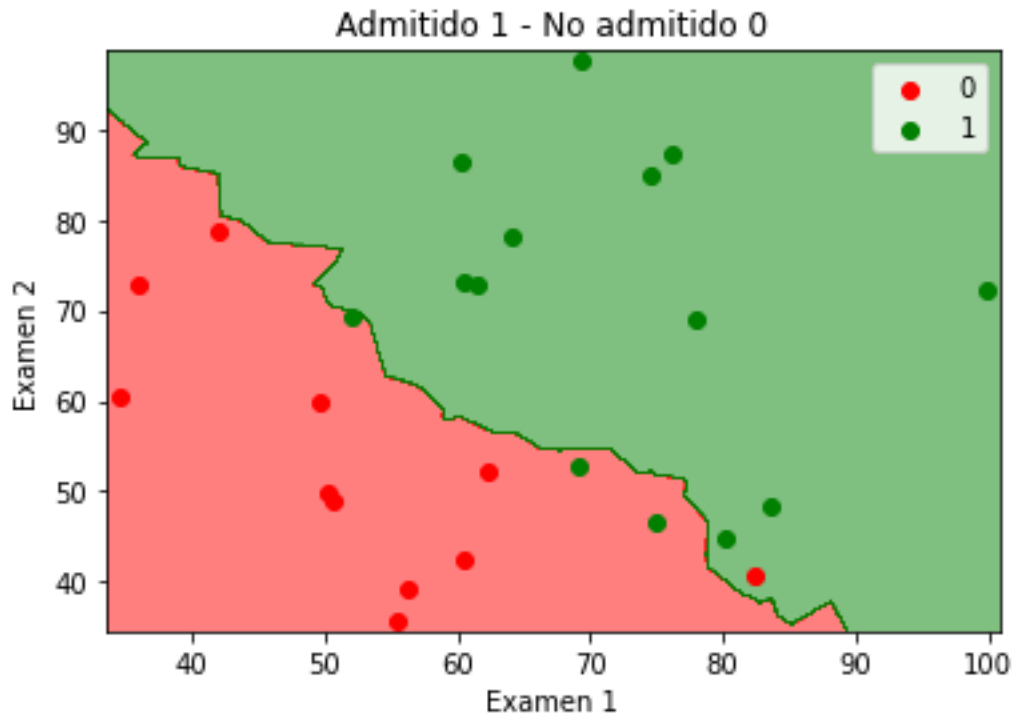


Predicho

GT	11	0
	2	12

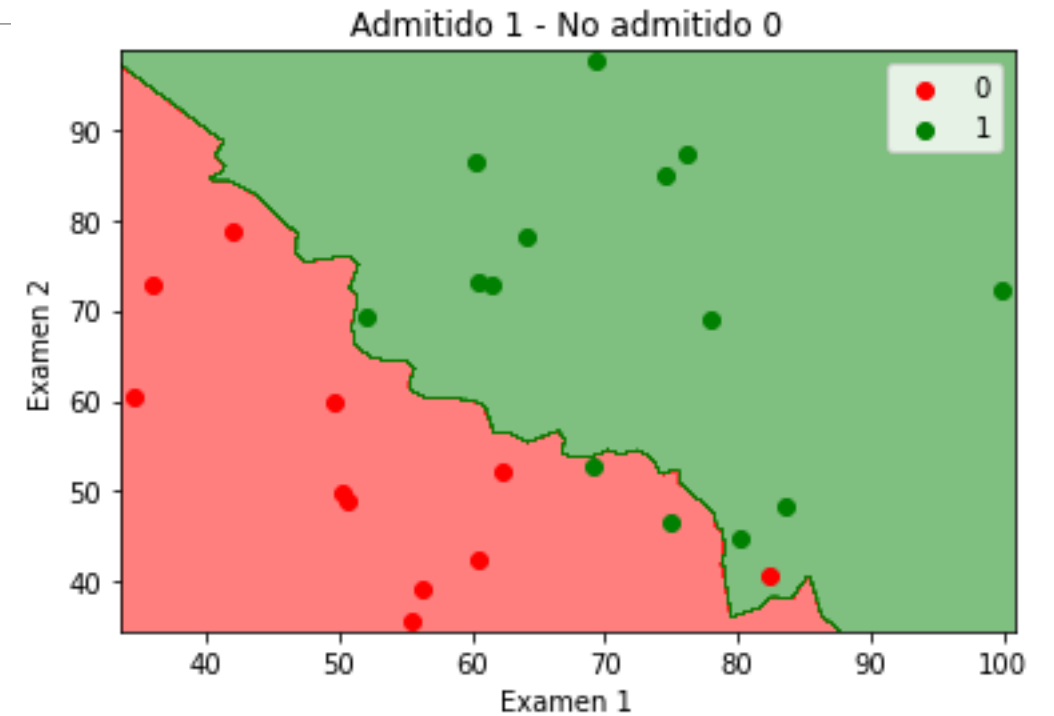
Resultado

K=5 Accuracy=0.84



Predicho	
GT	10
	1
GT	3
	11

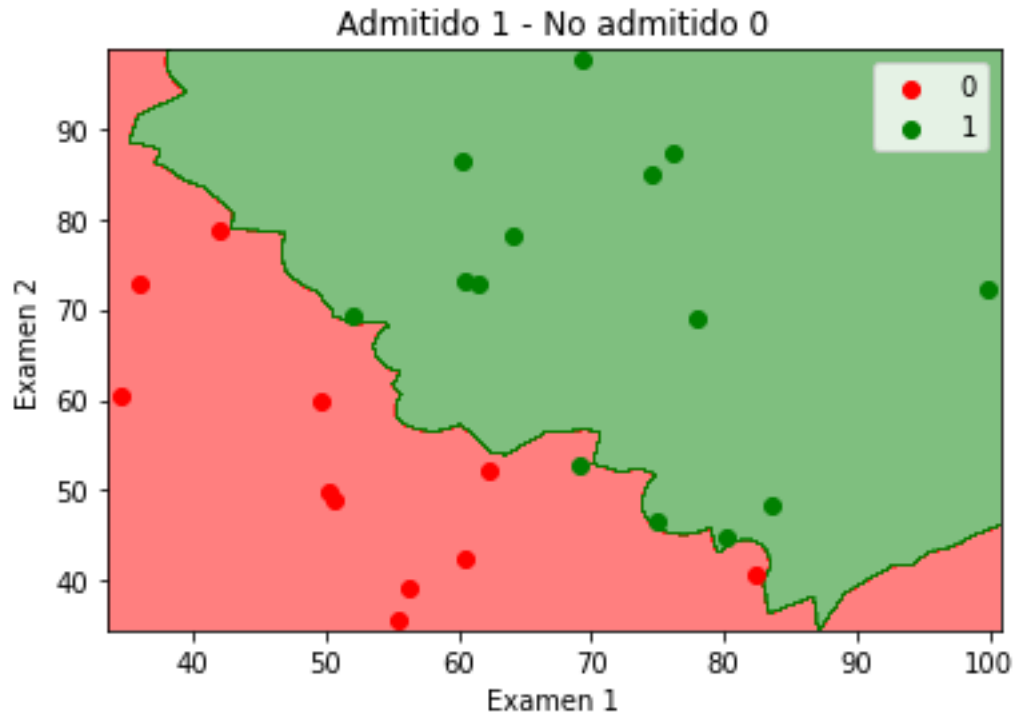
K=7 Accuracy=0.88



Predicho	
GT	10
	1
GT	2
	12

Resultado

K=3 Weights=distance

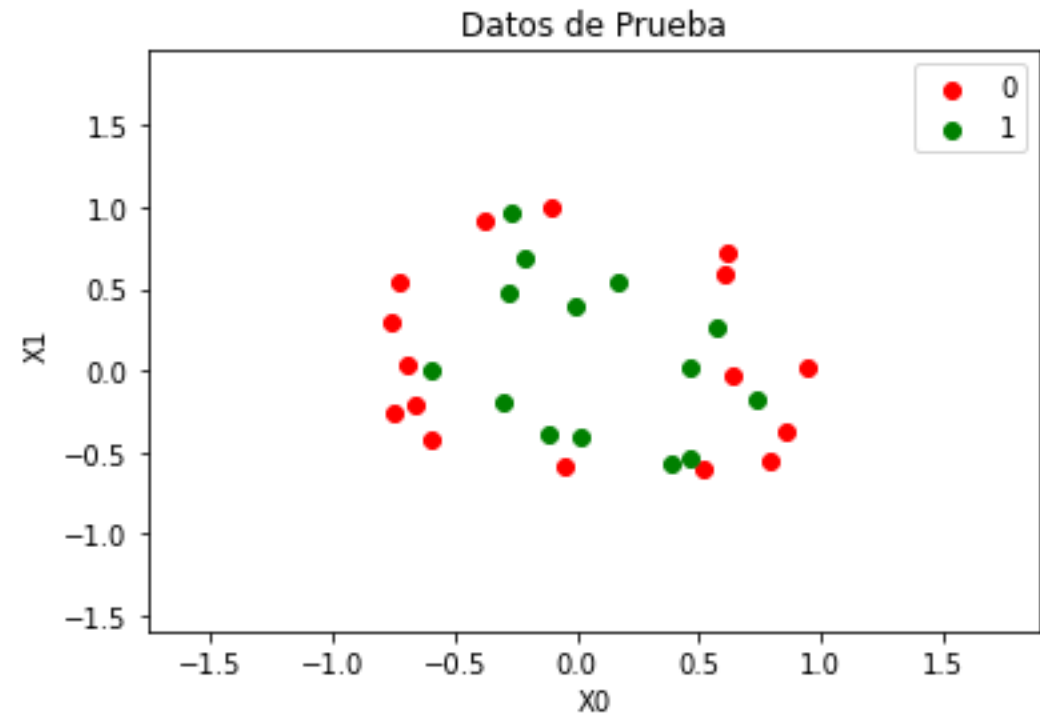
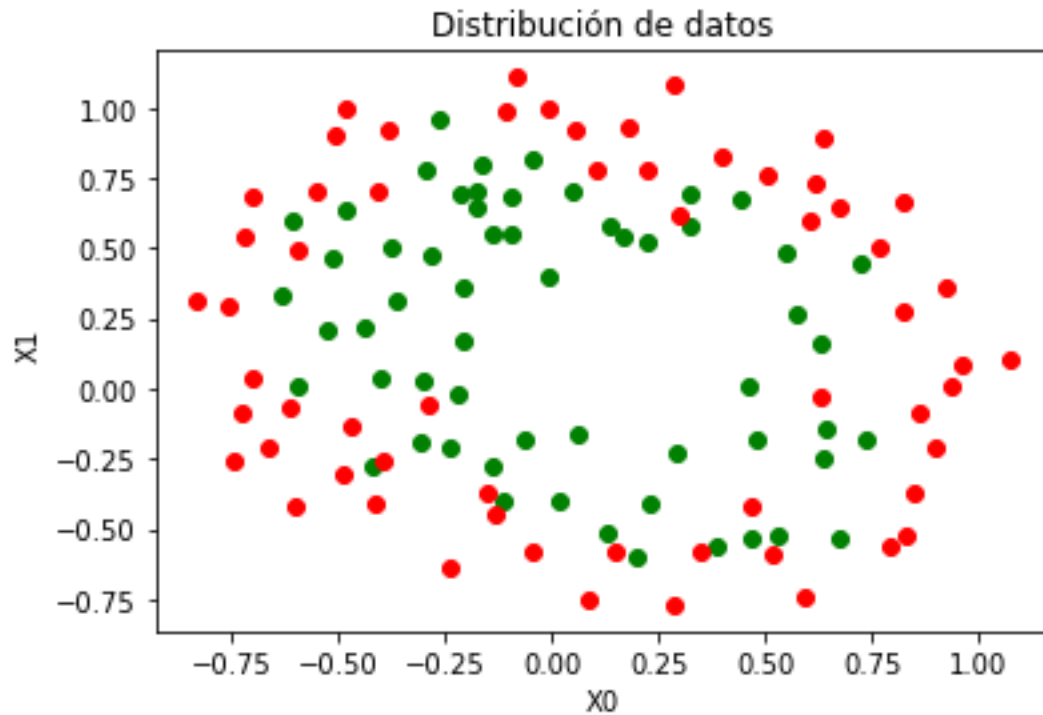


Accuracy=0.96

Predicho	
GT	11
	0
GT	1
	13

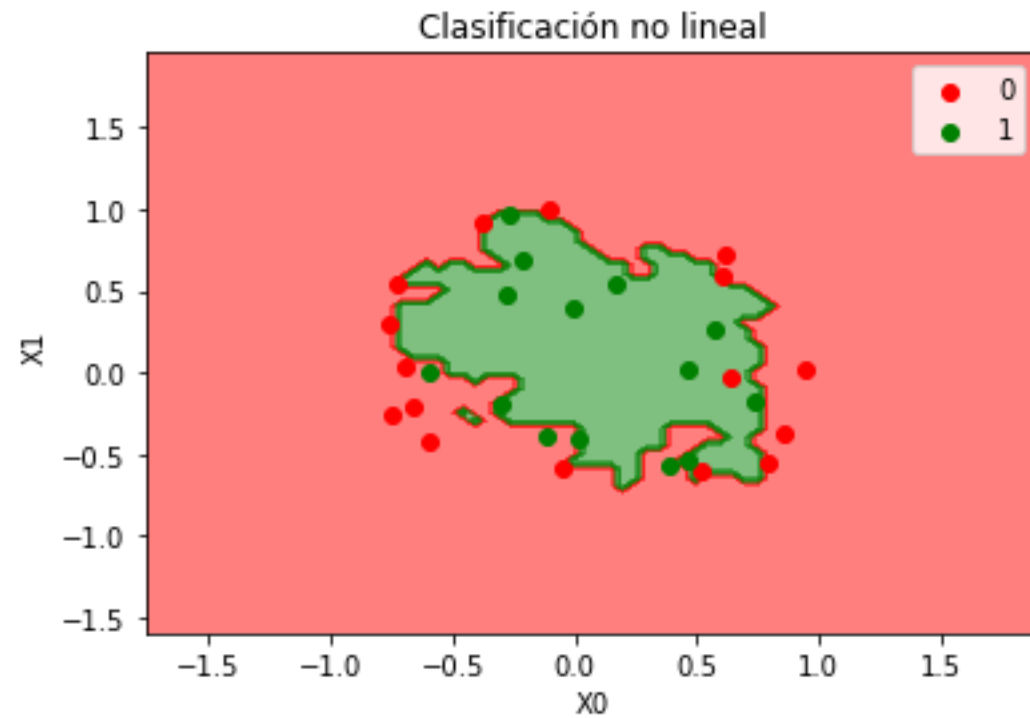
Implementación2: Frontera no lineal

Cambiando el número de vecinos, analiza el funcionamiento



Resultados

K=1



Accuracy= 0.77

Predicho	
GT	0
	1
0	13
1	3
0	4
1	10

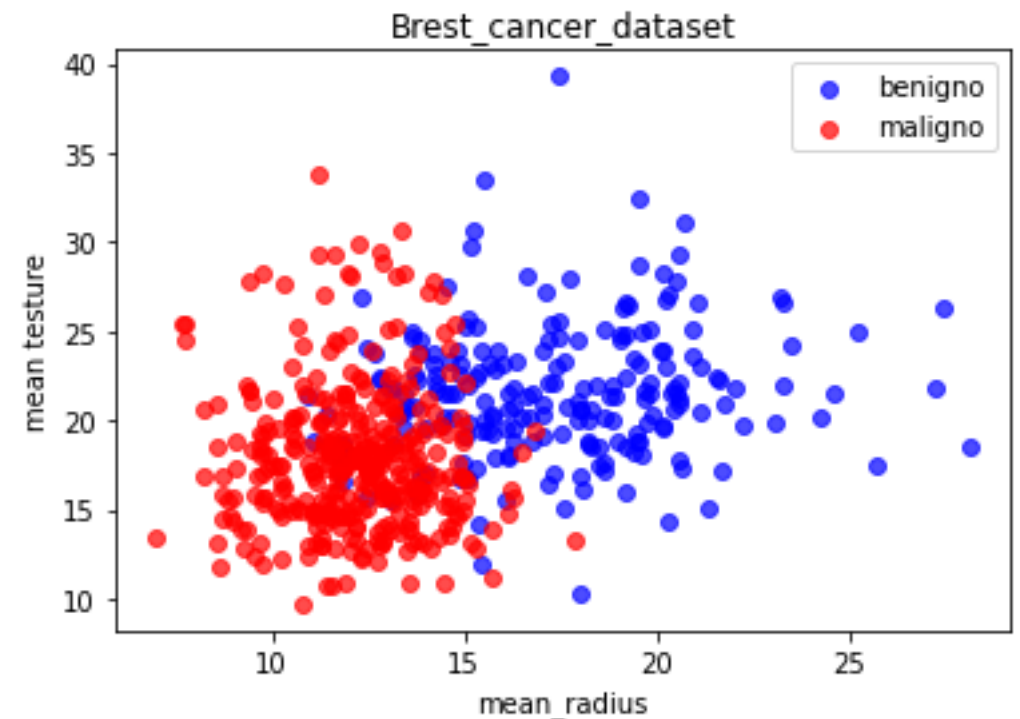
Implementación 3

Breast cancer wisconsin (diagnostic) dataset

-
- :Number of Instances: 569 (número de datos)
 - :Number of Attributes: 30 numeric, predictive attributes and the class
 - :Attribute Information:
 - radius (mean of distances from center to points on the perimeter)
 - texture (standard deviation of gray-scale values)
 - perimeter
 - area
 - smoothness (local variation in radius lengths)
 - compactness ($\text{perimeter}^2 / \text{area} - 1.0$)
 - concavity (severity of concave portions of the contour)
 - concave points (number of concave portions of the contour)
 - symmetry
 - fractal dimension ("coastline approximation" - 1)

The mean, standard error, and "worst" or largest (mean of the three worst/largest values) of these features were computed for each image, resulting in 30 features. For instance, field 0 is Mean Radius, field 10 is Radius SE, field 20 is Worst Radius.

- class:
- WDBC-Malignant
- WDBC-Benign



Resultado de clasificación

Máximo accuracy con k=3

92.31

47	6
5	85

Accuracy con k=1,5,7,9

91.81

46	7
5	85

Matriz de Confusión

Tarea : Aplicar clasificador de KNN a datos de Iris

Obtener Datafram usando sklearn

```
#### Obtener datos de Iris ####  
  
iris = load_iris()  
  
df = pd.DataFrame(iris.data, columns = iris.feature_names)  
  
df['target']=iris.target
```

- Usar todos los datos (4 características)
- Obtener resultados variando el número de vecinos
- Usando varias métricas para evaluar los resultados

Nota: puede usar paquetería Seaborn para dibujar “pairplot”

Base de datos de Iris



Setosa (0)



Versicolor (1)



Virginica (2)

Cada especie tiene 50 datos

Base de datos



Largo sépalo	Ancho sépalo	Largo pétalo	Ancho pétalo
(cm)	(cm)	(cm)	(cm)
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2
:	:	:	:
4.9	2.5	4.5	1.7
7.3	2.9	6.3	1.8
6.7	2.5	5.8	1.8
7.2	3.6	6.1	2.5

Distribución de datos

